این است که زمان‌های دسترسی حافظه ممکن است بر اساس سیاست‌های تعادل بار و خویشاوندی پردازنده، همین‌طور بر روی سیستم‌های NUMA فرق داشته باشند.

هرچند که اکنون سیستم‌های موبایل شامل معماری‌های چندهسته‌ای هستند، برخی سیستم‌ها هم اکنون با استفاده از هسته‌ها طراحی می‌شوند که مجموعه‌ی دستورالعمل یکسانی را اجرا می‌کنند، هرچند با تعابیر سرعت ساعت و مدیریت انرژی، اعم از توانمندی تنظیم مصرف انرژی یک هسته در نقطه‌ی بیکاری هسته فرق دارند. این قبیل سیستم‌ها به **چندپردازشی ناهمگن**[1] (**HMP**) معروف هستند. توجه داشته باشید که این شکلی از چندپردازشی نامتقارن، آن‌گونه که در بخش ۵-۵-۱ مطرح شد نیست که وظایف سیستم و کاربر، هر دو می‌توانند بر روی هر هسته‌ای اجرا شوند. بلکه، هدف پشت سر HMP بهترکردن مدیریت مصرف انرژی با گمارش وظایف به هسته‌های خاص بر اساس نیازهای خاص وظیفه می‌باشد.

این نوع معماری در پردازنده‌های ARM که از آن پشتیبانی می‌کنند به **big.LITTLE** معروف است که هسته‌های **بزرگ** با کارایی بالاتر با هسته‌های **کوچک** کارآمدی انرژی ترکیب می‌شوند. هسته‌های **بزرگ** انرژی بیشتری مصرف می‌کنند و لذا فقط می‌بایست برای دوره‌های زمانی کوتاهی استفاده شوند. همین‌طور، هسته‌های **کوچک** انرژی کمتری مصرف می‌کنند و لذا می‌توانند برای دوره‌های طولانی‌تری استفاده شوند.

مزایای متعددی در این رویکرد وجود دارد. با ترکیب تعدادی از هسته‌های کندتر با نمونه‌های سریع‌تر، یک زمان‌بند پردازنده می‌تواند وظایفی را که کارایی بالایی لازم ندارند، اما ممکن است برای دوره‌های طولانی‌تری اجرا شوند، (همچون وظایف پس‌زمینه) به هسته‌های کوچک بگمارد، که با این کار به حفظ شارژ باتری کمک می‌کند. با تشابهی، برنامه‌های محاوره‌ای که انرژی پردازشی بیشتری لازم دارند، اما ممکن است برای دوره‌های کوتاه‌تری اجرا شوند، می‌توانند به هسته‌های بزرگ گمارده شوند. به‌علاوه، اگر دستگاه موبایل در مد صرفه‌جویی انرژی باشد، هسته‌های بزرگی که عطش انرژی دارند می‌توانند غیرفعال شده و سیستم می‌تواند صرفاً به هسته‌های کوچک با انرژی کارآمد متکی باشد. ویندوز ۱۰ با اجازه‌دادن به یک نخ در انتخاب یک سیاست زمان‌بندی که با نیازهای مدیریت انرژی آن سازگاری بهتری دارد از زمان‌بندی HMP پشتیبانی می‌کند.

۵-۶ زمانبندی بلادرنگ پردازنده

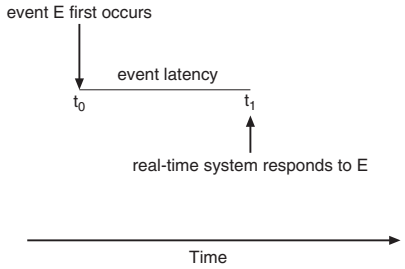
زمانبندی پردازنده برای سیستم‌های بلادرنگ مشمول ملاحظات خاصی است. در حالت کلی، می‌توانیم بین سیستم‌های بلادرنگ نرم و سیستم‌های بلادرنگ سخت تفاوت قایل شویم. **سیستم‌های بلادرنگ نرم** هیچ تضمینی نمی‌دهند که چه موقع، یک پردازه‌ی بلادرنگ بحرانی زمانبندی خواهد شد. فقط تضمین می‌کنند که پردازه نسبت به پردازه‌های غیربحرانی الویت خواهد گرفت. **سیستم‌های بلادرنگ سخت** نیازمندی‌های سخت‌گیرانه‌تری دارند. یک وظیفه بایستی تا ضرب‌الاجلی[2] که دارد سرویس داده شود؛ سرویس بعد از ضرب‌الاجل چنان است

1. Heterogeneous multiprocessing (HMP)
2. Deadline

که اصلاً هیچ‌گونه سرویسی صورت نگرفته است. در این بخش، ملاحظات متعدد مربوط به زمانبندی را در هر دوی سیستم‌های بلادرنگ نرم و سخت، بحث و بررسی می‌کنیم.

۵-۶-۱ کمینه‌سازی دیرکرد

طبیعت رویدادگرای یک سیستم بلادرنگ را در نظر بگیرید. سیستم نوعاً به انتظار یک رویداد است که به صورت بلادرنگ اتفاق می‌افتد. رویدادها ممکن است در نرم‌افزار–موقعی که یک تایمر سر می‌رسد –یا در سخت‌افزار–موقعی که یک خودرو با کنترل از دور تشخیص می‌دهد که به یک مانع نزدیک می‌شود– پیش بیایند. زمانی که رویدادی اتفاق می‌افتد، سیستم بایستی در اسرع وقت ممکن، آن را تشخیص داده و سرویس بدهد. مدت زمانی که از وقوع رویداد تا لحظه‌ی سرویس آن سپری می‌شود، **دیرکرد رویداد** نامیده می‌شود (شکل ۵–۱۷).



شکل ۵–۱۷ دیرکرد رویداد.

معمولاً، رویدادهای متفاوت، نیازمندی‌های دیرکرد متفاوتی دارند. برای مثال، نیازمندی دیرکرد یک سیستم ترمز ضد قفل ممکن است بین ۳ تا ۵ میلی‌ثانیه باشد. یعنی، از لحظه‌ای که خودرو برای بار اول تشخیص داد که سر می‌خورد، سیستم کنترل ترمزهای ضد قفل، ۳ تا ۵ میلی‌ثانیه فرصت پاسخ و کنترل شرایط را دارد. هرگونه پاسخی که زمان بیشتری طول بکشد ممکن است خودرو را دیگر از کنترل خارج کند. در مقایسه، سیستم تعبیه شده‌ی کنترل رادار در خطوط هواپیمایی ممکن است دوره‌ی دیرکرد چندین ثانیه را تحمل کند.

دو نوع دیرکرد، کارایی سیستم‌های بلادرنگ را متأثر می‌سازند که عبارتند از:

۱- دیرکرد وقفه

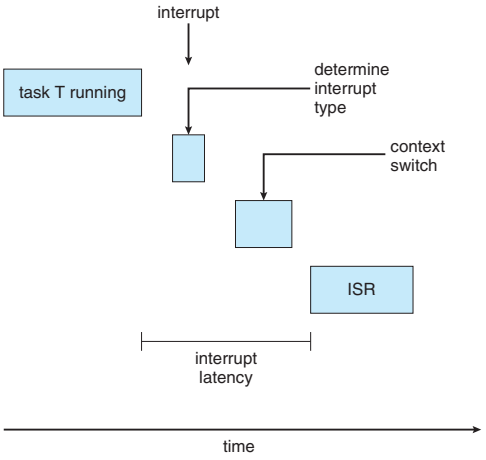
۲- دیرکرد اعزام

دیرکرد وقفه به دوره‌ی زمانی رسیدن یک وقفه در پردازنده تا شروع روال سرویس وقفه مربوط می‌شود. موقعی که یک وقفه رخ می‌دهد، سیستم‌عامل بایستی نخست دستورالعملی را که در حال اجراست تکمیل کرده و نوع وقفه‌ی اتفاق افتاده را تعیین کند. سپس، می‌بایست حالت پردازه‌ی فعلی را قبل از سرویس وقفه با استفاده از روال سرویس مخصوص وقفه (ISR) ذخیره نماید. کل زمانی که برای انجام این کارها لازم می‌شود دیرکرد وقفه است (شکل ۵–۱۸).

پر واضح است که در سیستم‌عامل‌های بلادرنگ مهم است که دیرکرد وقفه کمینه شود تا این اطمنیان حاصل شود که به وظایف بلادرنگ توجه آنی صورت می‌گیرد. در حقیقت، در

سیستم‌های بلادرنگ سخت، دیرکرد وقفه نمی‌تواند به سادگی کمینه شود، بایستی کراندار شود تا نیازمندی‌های محض این سیستم‌ها برآورده شوند.

یک عامل مهم که در دیرکرد وقفه نقش دارد، میزان زمان‌هایی است که ممکن است وقفه‌ها در حالی که ساختمان‌داده‌های هسته در حال به روزرسانی هستند، غیرفعال شوند. سیستم‌عامل‌های بلادرنگ لازم دارند که وقفه‌ها فقط برای دوره‌های زمانی بسیار کوتاه غیرفعال شوند.



شکل ۵-۱۸ دیرکرد وقفه.

میزان زمان لازم برای اعزام‌کننده‌ی زمانبندی در توقف یک پردازه و شروع دیگری، تحت عنوان **دیرکرد اعزام** معروف است. تدارک وظایف بلادرنگ با دسترسی آنی پردازنده حکم می‌کند که سیستم‌عامل‌های بلادرنگ، این دیرکرد را نیز کمینه سازند. کارآمدترین تکنیک پایین نگه داشتن دیرکرد اعزام، داشتن هسته‌های پس‌گرفتنی است. در سیستم‌های بلادرنگ سخت، دیرکرد اعزام نوعاً در حد چند میکروثانیه سنجیده می‌شود.

در شکل ۵-۱۹، آرایش دیرکرد اعزام را نشان داده‌ایم. **مرحله‌ی تضاد** دیرکرد اعزام دو مؤلفه دارد که عبارتند از:

۱- پس‌گیری هر پردازه‌ای که در هسته اجرا دارد

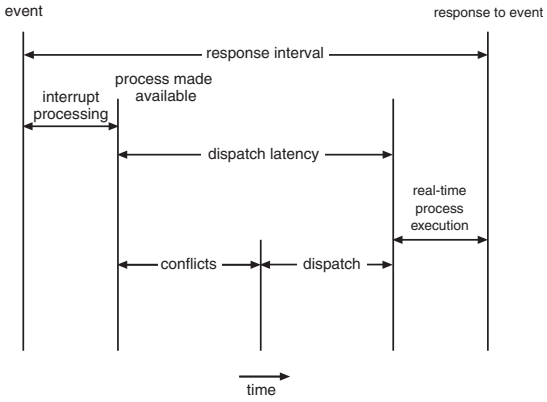
۲- رهاسازی منابع مورد نیاز پردازه‌ی با الویت بالا توسط پردازه‌ی با الویت پایین

با پیروی از مرحله‌ی تضاد، مرحله‌ی اعزام، پردازه‌ی با الویت بالا را به روی یک پردازنده‌ی در دسترس زمان‌بندی می‌نماید.

۵-۶-۲ زمانبندی الویتی

مهم‌ترین ویژگی یک سیستم‌عامل بلادرنگ، پاسخ‌گویی آنی به یک پردازه‌ی بلادرنگ، به محض نیاز پردازه به پردازنده می‌باشد. در نتیجه، زمانبند سیستم‌عامل بلادرنگ بایستی از یک الگوریتم الویتی پس‌گیر پشتیبانی کند. به یاد دارید که الگوریتم‌های زمانبندی الویتی، به هر

پردازه یک الویت بر اساس اهمیت آن نسبت می‌دهند؛ وظایف مهم‌تر به نسبت آنهایی که اهمیت کمتری دارند، الویت‌های بالاتری می‌گیرند. اگر زمانبند، پس‌گیری را نیز پشتیبانی کند، پردازه‌ای که فعلاً در پردازنده اجرا دارد در صورتی که یک پردازه‌ی با الویت بالاتری برای اجرا برسد، پس‌گرفته خواهد شد.



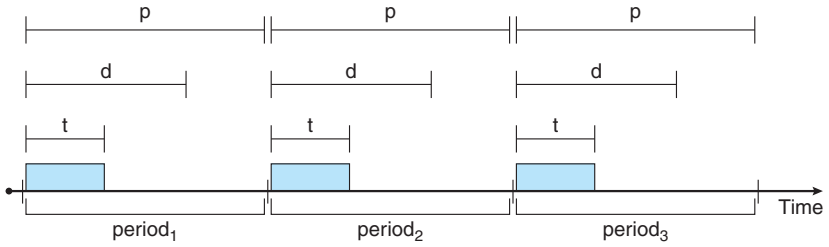
شکل ۵-۱۹ دیرکرد اعزام.

الگوریتم‌های زمانبندی الوینی پس‌گرفتنی با جزئیات بیشتر در بخش ۵-۳-۴ بحث می‌شوند و بخش ۵-۷، مثال‌هایی از امکانات زمانبندی بلادرنگ نرم سیستم‌عامل‌های لینوکس، ویندوز و سولاریس ارائه می‌دهد. هر کدام از این سیستم‌ها بالاترین الویت زمانبندی را به پردازه‌های بلادرنگ می‌دهند. برای مثال، ویندوز ۳۲ سطح الویت متفاوت دارد. بالاترین سطوح–مقادیر الویت ۱۶ تا ۳۱–برای پردازه‌های بلادرنگ محفوظ هستند. سولاریس و لینوکس نیز طرح‌های الویت مشابهی دارند.

توجه داشته باشید که فراهم‌ساختن یک زمانبند الویتی پس‌گرفتنی فقط کارکرد بلادرنگ نرم را تضمین می‌کند. سیستم‌های بلادرنگ سخت بایستی تضمین بیشتری بدهند تا وظایف بلادرنگ بر اساس نیازمندی‌های ضرب‌الاجل خود سرویس بگیرند و تضمین این چنینی مستلزم امکانات زمانبندی اضافه‌تری است. در باقی این بخش، الگوریتم‌های زمانبندی مناسب سیستم‌های بلادرنگ سخت را مطرح می‌کنیم.

با این همه، پیش از این که به جزئیات زمانبندهای خاصی بپردازیم، بایستی مشخصه‌های خاص پردازه‌هایی را که قرار است زمانبندی شوند تعریف کنیم. اولاً، پردازه‌ها **متناوب** محسوب می‌شوند. یعنی، پردازنده را در فواصل (دوره‌های) ثابتی لازم دارند. همین که یک پردازه‌ی متناوب پردازنده را بدست گرفت، یک زمان پردازش ثابت t، یک ضرب‌الاجل d که تا سپری‌شدن آن بایستی توسط پردازنده سرویس بگیرد و یک تناوب p دارد. رابطه‌ی زمان پردازش، ضرب‌الاجل و تناوب می‌تواند به صورت $0 \leq t \leq d \leq p$ بیان شود. **نرخ** یک وظیفه‌ی متناوب برابر $1/p$ می‌باشد. شکل ۵-۲۰، اجرای یک پردازه‌ی متناوب را به مرور زمان نشان می‌دهد. زمانبندها می‌توانند از این مشخصه‌ها بهره برده و الویت‌ها را بر اساس ضرب‌الاجل یک پردازه یا نیازمندی‌های نرخی تعیین کنند.

چیزی که با این شکل زمانبندی غیرعادی است این است که یک پردازه ممکن است مجبور به اعلان نیازمندی‌های ضرب‌الاجل خود به زمانبند شود. در ادامه، زمانبند با استفاده از تکنیکی تحت عنوان الگوریتم **کنترل پذیرش**، یکی از این دو کار را انجام می‌دهد. یا پردازه را با تضمین این که پردازه سر وقت تکمیل خواهد شد، پذیرفته و یا اگر نتواند تضمین بدهد که وظیفه تا رسیدن ضرب‌الاجلش سرویس خواهد گرفت، درخواست را ناممکن قلمداد کرده و رد می‌کند.



شکل ۵-۲۰ وظیفه‌ی متناوب.

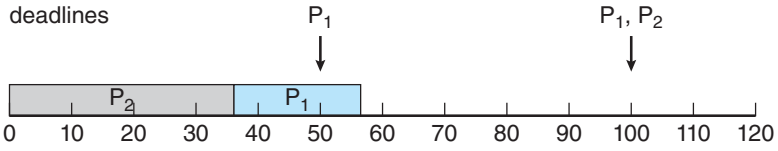
۵-۶-۳ زمانبندی نرخ-یکنواخت

الگوریتم زمانبندی **نرخ-یکنواخت**، وظایف متناوب را با استفاده از یک سیاست الویت پس‌گرفتنی ایستا زمانبندی می‌کند. اگر یک پردازه‌ی با الویت پایین در حال اجرا باشد و یک پردازه‌ی با الویت بالاتر آماده‌ی اجرا گردد، پردازه‌ی با الویت پایین‌تر را پس خواهد زد. هر وظیفه‌ی متناوب به محض ورود به سیستم، الویتی مطابق عکس تناوبش در اختیار می‌گیرد. تناوب کمتر، الویت بالاتری می‌گیرد؛ تناوب طولانی، الویت پایین‌تری می‌گیرد. استدلال پشت این سیاست این است که الویت بالاتر به وظایفی اعطا شود که پردازنده را در بیشتر موارد نیاز دارند. به علاوه، زمانبندی نرخ-یکنواخت فرض را بر این می‌گذارد که زمان پردازش یک پردازه‌ی متناوب برای هر یک از انفجارهای پردازنده یکسان است. یعنی، هر زمان که پردازه پردازنده را در اختیار می‌گیرد، طول انفجار پردازنده‌اش یکی است.

به این مثال توجه کنید. دو پردازه‌ی P_1 و P_2 را داریم. تناوب‌های P_1 و P_2، به ترتیب ۵۰ و ۱۰۰ می‌باشند-یعنی، $p_1 = 50$ و $p_2 = 100$. زمان‌های پردازش P_1 برابر $t_1 = 20$ و برای P_2 برابر $t_1 = 35$. ضرب‌الاجل هر پردازه لازم دارد که انفجار پردازنده‌اش را تا شروع تناوب بعدی تکمیل نماید.

بایستی در ابتدا این سؤال را از خود بپرسیم که آیا این امکان وجود دارد که این وظایف را چنان زمانبندی کرد که به ضرب‌الاجل خود هم برسند. اگر بهره‌وری پردازنده‌ی پردازه‌ی P_i را به صورت ضریب انفجار آن به تناوب آن-t_i/p_i-اندازه‌گیری کنیم، بهره‌وری پردازنده‌ی P_1 برابر $20/50 = 0.40$، P_2 برابر $35/100 = 0.35$ و کل بهره‌وری پردازنده برابر ۷۵ درصد خواهد بود. بنابراین، به نظر می‌رسد که می‌توانیم این وظایف را جوری زمانبندی کنیم که به ضرب‌الاجل خود نیز برسند و همچنان دورهایی را برای پردازنده باقی بگذاریم.

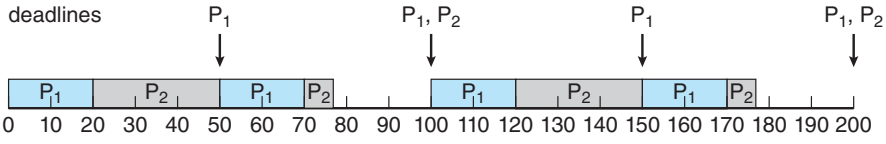
فرض کنید به P_2 الویت بالاتری از P_1 بدهیم. اجرای P_1 و P_2 با این شرایط در شکل ۵-۲۱ نشان داده می‌شود. همان‌گونه که می‌بینیم، P_2 اجرای خود را اول شروع کرده و در زمان ۳۵ تکمیل می‌شود. در اینجا، P_1 کار خود را شروع کرده و انفجار پردازنده‌اش را در زمان ۵۵ کامل می‌کند. با این همه، اولین ضرب‌الاجل P_1 در زمان ۵۰ بود، لذا زمانبند باعث شده است که P_1 ضرب‌الاجل خود را از دست بدهد.



شکل ۵-۲۱ زمانبندی وظیفه‌ها موقعی که P_2 دارای الویت بالاتری از P_1 است.

حال فرض کنید که از زمانبندی نخ-یکنواخت استفاده می‌کنیم که در آن به P_1 الویت بالاتری نسبت به P_2 می‌دهیم چرا که تناوب P_1 از P_2 کوتاهتر است. اجرای این پردازه‌ها با این شرایط در شکل ۵-۲۲ نشان داده می‌شود. P_1 اول شروع شده و انفجار پردازنده‌اش را در زمان ۲۰ تکمیل می‌کند، با این کار به ضرب‌الاجل نخست خود می‌رسد. در اینجا، P_2 اجرایش را شروع می‌کند و تا زمان ۵۰ اجرا می‌شود. در این لحظه، توسط P_1 پس‌گرفته می‌شود، هرچند که هنوز ۵ میلی‌ثانیه از انفجار پردازنده‌اش باقی است. P_1 انفجار پردازنده‌اش را در زمان ۷۰ کامل می‌کند که در آن لحظه، زمانبند P_2 را از سر می‌گیرد. P_2 انفجار پردازنده‌اش را در زمان ۷۵ تکمیل می‌کند که به ضرب‌الاجل نخست خود نیز می‌رسد. سیستم تا لحظه‌ی ۱۰۰ بیکار است، که P_1 دوباره زمانبندی می‌شود.

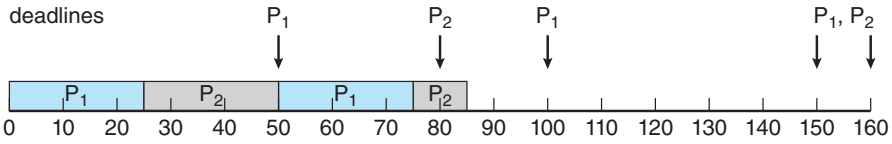
زمانبندی نرخ-یکنواخت از این نظر بهینه به حساب می‌آید که اگر یک مجموعه پردازه نتواند توسط این الگوریتم زمانبندی شود، نمی‌تواند با هیچ الگوریتم دیگری که الویت‌های ایستا می‌دهد زمانبندی شود. باز بیایید یک مجموعه پردازه را که نمی‌تواند با استفاده از الگوریتم نرخ-یکنواخت زمانبندی شود بررسی کنیم.



شکل ۵-۲۲ زمانبندی نرخ-یکنواخت.

فرض کنید پردازه‌ی P_1 دارای تناوب $p_1 = 50$ و انفجار پردازنده‌ی $t_1 = 25$ باشد. مقادیر متناظر P_2 عبارتند از $p_2 = 80$ و $t_2 = 35$. زمانبندی نرخ-یکنواخت، الویت بالاتر را به P_1 خواهد داد، چرا که تناوب کوتاهتری دارد. بهره‌وری کل پردازنده برای دو پردازه، $(35/80) + (25/50) = 0.94$ می‌شود و لذا منطقاً به نظر می‌رسد که دو پردازه بتوانند زمانبندی شده و هنوز یک زمان ۶ درصد هم برای پردازنده باقی بگذارند. شکل ۵-۲۳، زمانبندی پردازه‌های P_1 و P_2 را نشان می‌دهد. در ابتدا، P_1 اجرا می‌شود تا این که انفجار پردازنده‌اش را در زمان ۲۵ تکمیل می‌کند. سپس، پردازه‌ی P_2 شروع به

اجرا کرده و تا زمان ۵۰ اجرا می‌شود که در این لحظه، توسط P_1 پس‌گرفته می‌شود. در اینجا، P_2 هنوز ۱۰ میلی‌ثانیه از انفجار پردازنده‌اش باقی است. پردازه‌ی P_1 تا زمان ۷۵ اجرا می‌شود؛ در نتیجه، P_2 بعد از ضرب‌الاجل تکمیل زمان انفجار پردازنده‌اش در زمان ۸۰، انفجار پردازنده‌اش را در زمان ۸۵ به اتمام می‌رساند.



شکل ۵-۲۳ از دست رفتن ضرب‌الاجل‌ها با زمانبندی نرخ-یکنواخت.

پس، زمانبندی نرخ-یکنواخت به رغم بهینه‌بودن یک محدودیت دارد: بهره‌وری پردازنده کراندار است و همواره این امکان وجود ندارد که بهره‌وری منابع پردازنده به طور کامل بیشینه گردند. بهره‌وری بدترین حالت پردازنده برای زمانبندی N پردازه به صورت زیر است:

$$N\,(2^{1/N} - 1).$$

بهره‌وری با داشتن یک پردازه در سیستم، ۱۰۰ درصد است، اما با افزایش تعداد پردازه‌ها و میل به سمت بی‌نهایت، این میزان تقریباً به ۶۹ درصد افت می‌کند. بهره‌وری پردازنده با دو پردازه به ۸۳ درصد محدود می‌شود. بهره‌وری ترکیبی پردازنده برای دو پردازه‌ی زمانبندی شده در شکل ۵-۲۱ و شکل ۵-۲۲ برابر ۷۵ درصد است؛ از این رو، تضمین است که الگوریتم زمانبندی نرخ-یکنواخت بتواندآنها را چنان زمانبندی کند که به ضرب‌الاجل خود نیز برسند. بهره‌وری ترکیبی برای دو پردازه‌ی زمانبندی شده در ۵-۲۳، تقریباً ۹۴ درصد است؛ از این رو، زمانبندی نرخ-یکنواخت نمی‌تواند تضمین بدهد که آنها بتوانند طوری زمانبندی شوند که به ضرب‌الاجل خود هم برسند.

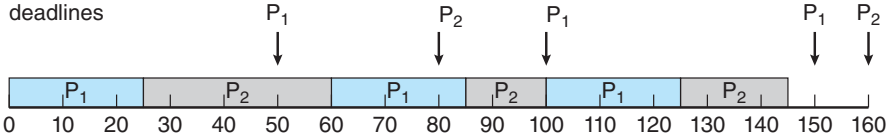
۵-۶-۴ زمانبندی قدیمی‌ترین-ضرب‌الاجل-اول

زمانبندی **قدیمی‌ترین-ضرب‌الاجل-اول**[1] (**EDF**) الویت‌ها را به‌صورت پویا مطابق ضرب‌الاجل تعیین می‌کند. هرچه ضرب‌الاجل قدیمی‌تر باشد، الویت بالاتری می‌گیرد؛ ضرب‌الاجل اخیرتر به الویت پایین‌تر می‌رسد. با سیاست EDF، موقعی که پردازه‌ای قابلیت اجرا پیدا می‌کند، بایستی نیازمندی‌های ضرب‌الاجل خود را به سیستم اطلاع بدهد. ممکن است جهت اعمال ضرب‌الاجل پردازه‌ای که به تازگی اجرایی شده است، الویت‌ها تنظیم شوند. توجه دارید که این رویه با زمانبندی نرخ-یکنواخت فرق دارد که در آنجا الویت‌ها ثابت هستند.

برای روشن‌شدن زمانبندی EDF، باز هم پردازه‌های نشان داده شده در شکل ۵-۲۳ را زمانبندی می‌کنیم که نتوانستند نیازمندی‌های ضرب‌الاجل خود را با زمانبندی نرخ-یکنواخت ارضا کنند. به خاطر دارید که P_1، مقادیر $p_1 = 50$ و $t_1 = 25$ و P_2، مقادیر $p_2 = 80$ و $t_2 = 35$ را داشتند. زمانبندی EDF برای این پردازه‌ها در ۵-۲۴ نشان داده می‌شود. پردازه‌ی P_1 قدیمی‌ترین

1. Earliest-deadline-first

ضرب‌الاجل را دارد، لذا الویت اولیه‌ی آن از پردازه‌ی P_2 بالاتر است. پردازه‌ی P_2 اجرایش را در انتهای انفجار پردازه‌ی P_1 شروع می‌کند. با این وجود، در حالی‌که زمانبندی نرخ-یکنواخت به P_1 اجازه می‌داد تا P_2 را در شروع تناوب بعدیش در زمان ۵۰ پس بگیرد، زمانبندی EDF به پردازه‌ی P_2 اجازه می‌دهد که به اجرایش ادامه بدهد. اکنون P_2 الویت بالاتری نسبت به P_1 دارد، چون ضرب‌الاجل بعدی آن (در زمان ۸۰) قدیمی‌تر از مال P_1 (در زمان ۱۰۰) است. بنابراین، هر دوی P_1 و P_2 به ضرب‌الاجل‌های اول خود می‌رسند. پردازه‌ی P_1 دوباره اجرای خود را در زمان ۶۰ شروع کرده و انفجار پردازنده‌ی دومش را در زمان ۸۵ تکمیل می‌کند، که به ضرب‌الاجل دوم خود در زمان ۱۰۰ هم می‌رسد. P_2 اجرایش را در این لحظه شروع می‌کند و فقط توسط P_1 در شروع تناوب بعدیش در زمان ۱۰۰ پس گرفته می‌شود. P_2 پس گرفته می‌شود، چون P_1 ضرب‌الاجل قدیمی‌تری (زمان ۱۵۰) نسبت به P_2 (زمان ۱۶۰) دارد. در زمان ۱۲۵، P_1 انفجار پردازنده‌اش را کامل می‌کند و P_2 اجرایش را از سر گرفته و با اتمام در زمان ۱۴۵، به ضرب‌الاجل خود هم می‌رسد. سیستم تا زمان ۱۵۰ بیکار است، موقعی که P_1 برای اجرای مجدد زمانبندی می‌شود.



شکل ۵-۲۴ زمانبندی قدیمی‌ترین-ضرب‌الاجل-اول.

برعکس الگوریتم نرخ-یکنواخت، زمانبندی EDF لازم ندارد که پردازه‌ها متناوب باشند، اجباری هم نیست که یک پردازه در هر انفجار پردازنده‌اش، یک میزان ثابت از وقت پردازنده را بگیرد. تنها نیازمندی این است که پردازه بایستی ضرب‌الاجل خود را موقع آمادگی برای اجرا اطلاع بدهد. جذابیت زمانبندی EDF این است که از نظر تئوری بهینه است-به صورت تئوری می‌تواند پردازه‌ها را چنان زمانبندی کند که هر پردازه به نیازمندی‌های ضرب‌الاجل خود رسیده و بهره‌وری پردازنده نیز ۱۰۰ درصد باشد. با این وجود، در عمل، به خاطر هزینه‌ی برگردان متن میان پردازه‌ها و رسیدگی به وقفه، رسیدن به این سطح از بهره‌وری پردازنده ناممکن است.

۵-۶-۵ زمانبندی سهم متناسب

زمانبندهای سهم **متناسب**[1] با تخصیص T سهم مابین تمامی برنامه‌ها عملیات دارند. یک برنامه می‌تواند N سهم زمان دریافت کند، بنابراین، تضمین است که برنامه ضریب N/T کل زمان پردازنده را در اختیار خواهد داشت. به عنوان یک مثال، فرض کنید که کل T = 100 سهم، مابین سه پردازه‌ی A، B و C تقسیم می‌شود. به A، ۵۰ سهم، به B، ۱۵ سهم و به C، ۲۰ سهم اعطا می‌شود. این طرح تضمین می‌کند که A، ۵۰ درصد، B، ۱۵ درصد و C، ۲۰ درصد از کل زمان پردازنده را در اختیار خواهند داشت.

1. Proportional share

زمانبندهای سهم متناسب بایستی به همراه یک سیاست کنترل پذیرش کار کنند تا تضمین شود که یک برنامه سهم زمانی تخصیصی خود را دریافت می‌کند. سیاست کنترل پذیرش، تنها در صورتی درخواست کارفرمایی را برای تعدادی سهم خواهد پذیرفت که سهم کافی موجود باشد. در مثال جاری، ۸۵ = ۲۰ + ۱۵ + ۵۰ سهم را از کل ۱۰۰ سهم تخصیص داده‌ایم. اگر پردازه‌ی جدید D درخواست ۳۰ سهم داشته باشد، کنترلر پذیرش از ورود D به سیستم ممانعت می‌کند.

۵-۶-۶ زمانبندی بلادرنگ پوزیکس

استاندارد پوزیکس هم گسترس‌هایی برای محاسبات بلادرنگ فراهم می‌کند–POSIX.1b. در اینجا، تعدادی API پوزیکس مربوط به زمانبندی نخ‌های بلادرنگ را مطرح می‌کنیم. پوزیکس دو رده‌ی زمانبندی برای نخ‌های بلادرنگ تعریف می‌کند که عبارتند از:

- SCHED_FIFO
- SCHED_RR

SCHED_FIFO نخ‌ها را مطابق سیاست اولین–ورود، اول–سرویس، با استفاده از یک صف FIFO آن‌گونه که در بخش ۵-۳-۱ مطرح شد، زمانبندی می‌کند. با این وجود، هیچ‌گونه برش زمانی مابین نخ‌های با الویت یکسان وجود ندارد. بنابراین، پردازنده به نخ بلادرنگ با بالاترین الویت در جلوی صف FIFO اعطا خواهد شد تا این که خاتمه بپذیرد یا مسدود شود. SCHED_RR از سیاست نوبت–چرخشی استفاده می‌کند. آن شبیه به SCHED_FIFO است جز این که مابین نخ‌های با الویت یکسان، برش زمانی فراهم می‌کند. پوزیکس یک رده‌ی زمانبندی دیگر– SCHED_OTHER–فراهم می‌کند، اما پیاده‌سازی آن تعریف نشده و خاص سیستم است؛ آن ممکن است روی سیستم‌های متفاوت به شیوه‌ای متفاوت رفتار کند.

API پوزیکس دو تابع زیر را برای خواندن و انتخاب سیاست زمانبندی مشخص می‌کند:

- pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)
- pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)

اولین پارامتر هر دو تابع، اشاره‌گری به مجموعه صفات نخ است. پارامتر دوم، یا (۱) اشاره‌گری به یک متغیر صحیح است که برابر سیاست زمانبندی جاری (با تابع pthread_attr_getsched_policy()) انتخاب می‌شود (۲) و یا یک مقدار صحیح (SCHED_FIFO، SCHED_RR، یا SCHED_OTHER) برای تابع pthread_attr_setsched_policy() می‌باشد. هر دو تابع با وقوع خطا، مقادیر غیرصفر بر می‌گردانند.

در شکل ۵-۲۵، یک برنامه‌ی نخ پوزیکس را با استفاده از این API نشان می‌دهیم. این برنامه در ابتدا سیاست زمانبندی جاری را تعیین کرده و سپس الگوریتم زمانبندی را برابر SCHED_FIFO انتخاب می‌کند.

۵-۷ مثال‌های سیستم‌عاملی

باز هم به توصیف سیاست‌های زمانبندی سیستم‌عامل‌های لینوکس، ویندوز و سولاریس بر می‌گردیم. توجه به این نکته مهم است که در اینجا از اصطلاح زمانبندی پردازه با مفهوم کلی